UNIVERSIDADE DE TRÁS-OS-MONTES E ALTO DOURO

Licenciatura em Engenharia Informática

Sistemas Distribuídos

Trabalho Prático 1

Hugo Paredes | Tiago Pinto | Cristiano Pendão

João Nogueira al81605

Luis Carvalho al81967

Vila Real, abril 2026

1. Introdução

O conceito *One Health* estabelece a forte interligação entre a saúde humana, animal e ambiental, sublinhando a importância da monitorização de fatores urbanos, como a qualidade do ar, a temperatura, a humidade e o ruído, para a deteção de indicadores relevantes na identificação de riscos para a saúde pública.

Para dar resposta a este desafio epidemiológico, foi concebido um sistema distribuído em três camadas que simula uma infraestrutura de monitorização ambiental urbana. A arquitetura adotada é composta por Sensores, que atuam como dispositivos locais de recolha de dados nas mais variadas zonas da cidade; *Gateways*, que funcionam como nós intermédios para agregação, validação e encaminhamento na periferia da rede; e um Servidor central, responsável pelo armazenamento, processamento e análise de toda a informação recebida. Este relatório documenta as opções arquitetónicas, o protocolo de rede desenhado e as soluções de concorrência adotadas para garantir um fluxo de dados fiável e contínuo.

2. Protocolo de Mensagens

Para gerir os diálogos entre o SENSOR, o GATEWAY e o SERVIDOR, foi desenhado um protocolo de comunicação assente na utilização de *sockets* sobre TCP/UDP em linguagem C#. De forma a garantir a flexibilidade na transmissão de métricas tão diversas como temperatura ou fluxos de vídeo em tempo real, concebeu-se uma estrutura de mensagem universal dividida num cabeçalho descritivo em formato de texto e num corpo de dados (*payload*) opcional em formato binário.

Cada mensagem transmitida contém atributos fundamentais no seu cabeçalho: o comando da operação (CMD), o identificador único do sensor (SID), o identificador do *gateway* (GID) e a *timestamp*, essencial para que as medições possam ser futuramente cruzadas e associadas ao período de exposição da população. O cabeçalho suporta também um dicionário flexível de dados, permitindo a injeção de parâmetros dinâmicos como a zona geográfica ou o tipo de métrica analisada. O final do cabeçalho de texto é estritamente demarcado por um terminador explícito ("<EOF>"), uma escolha crucial que garante a delimitação correta dos pacotes e previne a corrupção de leitura na *stream* TCP.

O fluxo de comunicação obedece a um ciclo de vida rigoroso. Após iniciar, o SENSOR estabelece ligação com o *GATEWAY* e identifica-se enviando os tipos de dados ambientais que é capaz de recolher. O GATEWAY, munido de ficheiros locais de configuração, verifica se o sensor se encontra devidamente registado e ativo para aquela zona. Validada a ligação, o sensor começa a enviar as suas medições de forma periódica. Para garantir a tolerância a falhas, o protocolo exige que o sensor envie periodicamente uma mensagem de "*heartbeat*"; caso o gateway deixe de receber este sinal durante um determinado período de tempo, o estado do sensor é atualizado para indisponível. Por fim, as entidades asseguram um comando de encerramento, terminando corretamente a comunicação libertando os recursos de rede em ambas as pontas.

3. Escolhas de Implementação

O desenvolvimento deste sistema distribuído obrigou a várias decisões técnicas orientadas à fiabilidade e à capacidade de escalabilidade. Embora o guião original recomendasse a gravação dos dados diretamente em ficheiros através do uso de *mutexes* para garantir o acesso sequencial, a equipa optou por cumprir a funcionalidade extra proposta e implementou uma base de dados relacional *SQLite* no servidor. Esta escolha abandonou a rigidez dos ficheiros físicos, garantindo transações seguras, a prevenção de perdas por concorrência e facilitando consultas avançadas por parte de sistemas externos.

No que toca à gestão da concorrência no SERVIDOR e no *GATEWAY*, ferramentas vitais para receber pedidos de múltiplos sensores em simultâneo, decidiu-se utilizar semáforos em detrimento dos *mutexes* clássicos sugeridos. A preferência pelos semáforos justificou-se pela sua capacidade de limitar e gerir o número máximo de ligações ativas na *thread pool*. Enquanto um *mutex* restringiria demasiadamente o sistema bloqueando a operação a apenas uma *thread* por ficheiro/recurso num dado momento, o semáforo atua como um regulador de tráfego, evitando a exaustão de memória sob picos de carga sem estrangular a natureza assíncrona das ligações TCP.

Ao nível da resiliência, o *GATEWAY* foi equipado com um gestor de cache local suportado no padrão "*Store-and-Forward*". Se a ligação entre o *gateway* e o servidor central da *cloud* for perdida temporariamente, o *gateway* não descarta as leituras vitais; este armazena os dados localmente, numa base de dados *SQLite*, reestabelecendo a sincronização automática e o envio pendente assim que a ligação à rede for restabelecida.

Finalmente, a arquitetura foi modernizada para um cenário de produção real. O servidor central e a base de dados foram encapsulados através de contentores Docker, garantindo a rápida replicação de ambientes. O *deployment* deste sistema em contentor foi realizado numa instância EC2 alojada na *cloud* da Amazon Web Services (AWS), acompanhado pelo desenvolvimento de uma interface Web em tempo real. Este *dashboard* consome diretamente os dados da base de dados e permite monitorizar a evolução ambiental de forma gráfica, maximizando o propósito do projeto de servir os processos de diagnóstico da cidade.

4. Conclusão e Trabalho Futuro

O projeto *UrbanHealth* respondeu com êxito a todos os requisitos arquitetónicos e operacionais definidos para a infraestrutura de simulação ambiental, destacando-se positivamente na gestão do processamento concorrente e no encaminhamento fiável de tráfego. O protocolo de comunicação flexível demonstrou grande solidez na gestão dinâmica de dados estáticos e binários. A implementação de uma cache intermédia aliada à utilização de semáforos e a uma base de dados robusta dotou o sistema de uma elevada tolerância a falhas e capacidade de integração contínua.

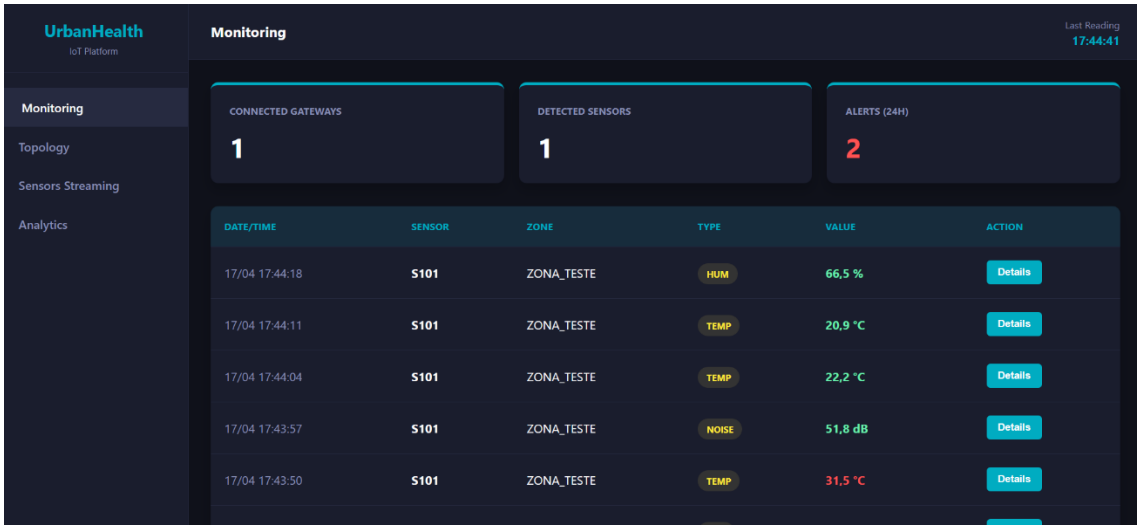
Como propostas de desenvolvimento futuro, podemos citar a integração de certificados TLS nos *sockets* para encriptar as transmissões, assegurando a privacidade da informação enviada.

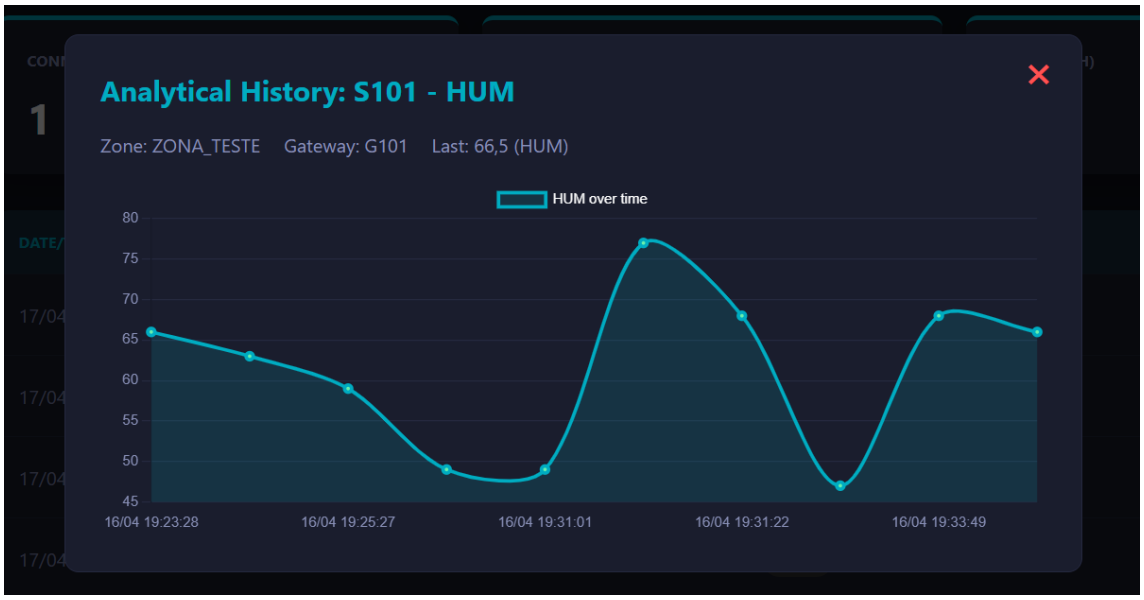
No campo analítico, pretende-se explorar a implementação de modelos preditivos de *Machine Learning* para a análise avançada dos dados recolhidos por cada zona urbana. A utilização destes modelos permitirá definir, de forma dinâmica e inteligente, os limiares ideais para a monitorização ambiental, otimizando significativamente a precisão na deteção de anomalias e a eficácia no lançamento de avisos preventivos à população.

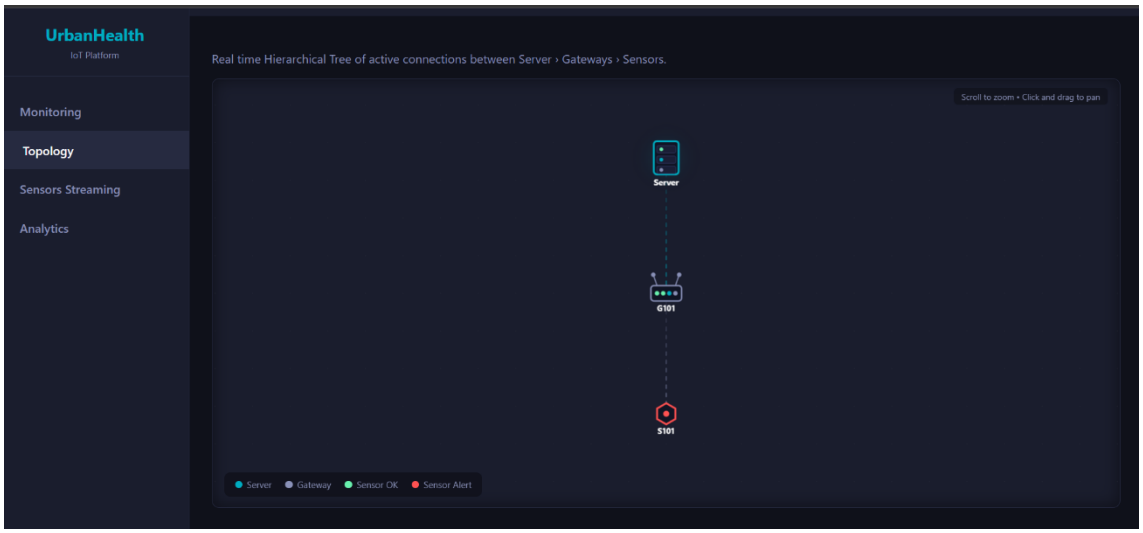
De igual modo, o servidor poderá evoluir para uma arquitetura de microsserviços gerida por um orquestrador como o *Kubernetes*, garantindo a elasticidade necessária para acompanhar o crescimento do tráfego de dados nas cidades modernas.

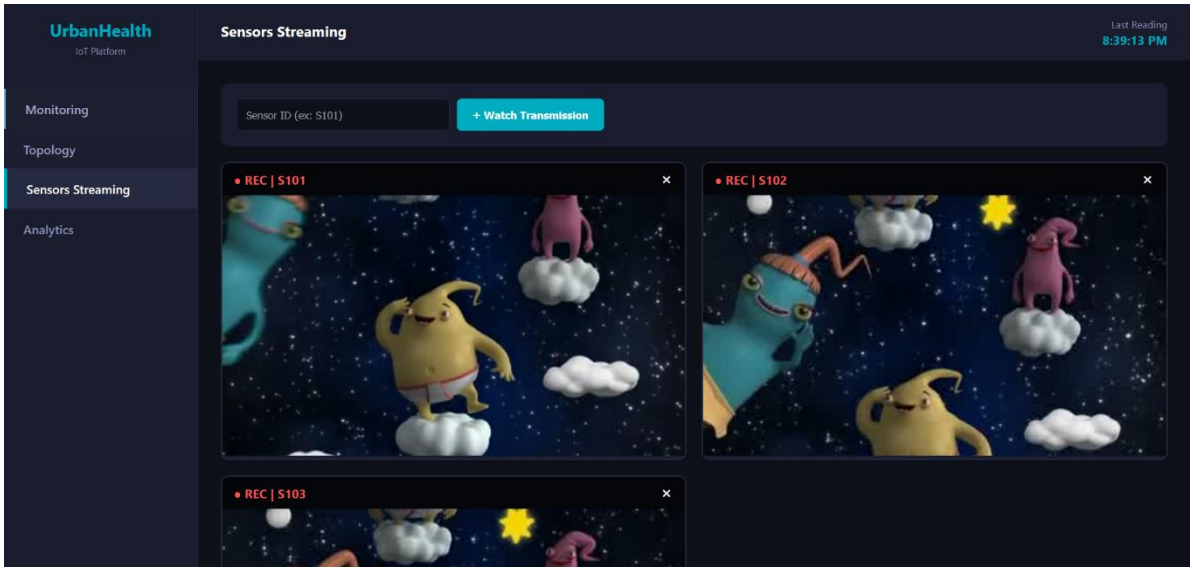
5. Anexos

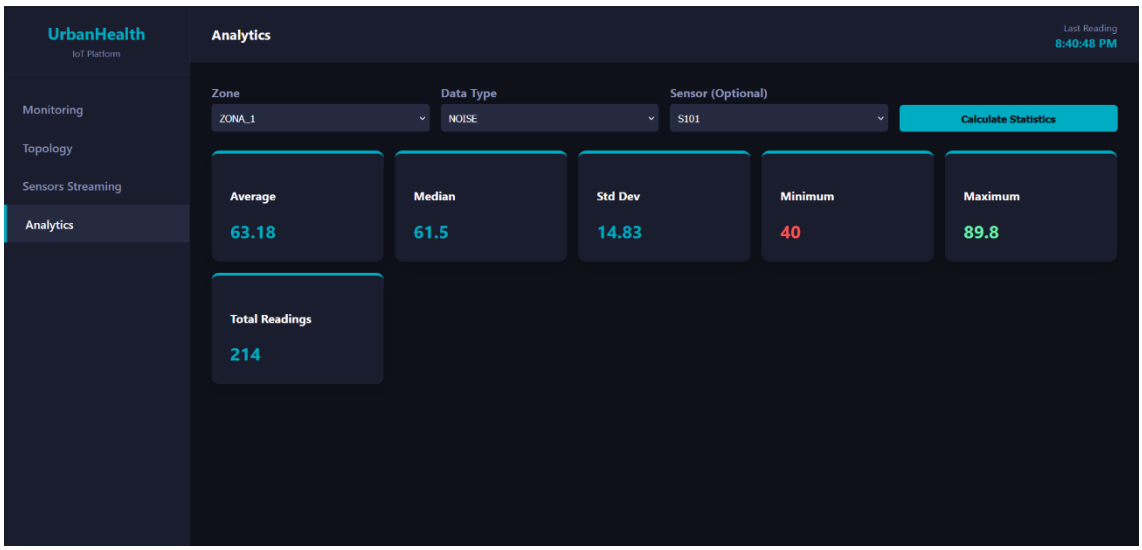
Repo: https://github.com/JoaoNogueiraGit/SD-TP1






UrbanHealth
IoT Platform
Monitoring
Topology
Sensors Streaming
Analytics
Real time Hierarchical Tree of active connections between Server › Gateways › Sensors.
Scroll to zoom • Click and drag to pan
Server
G101
S101
Server
Gateway
Sensor OK
Sensor Alert

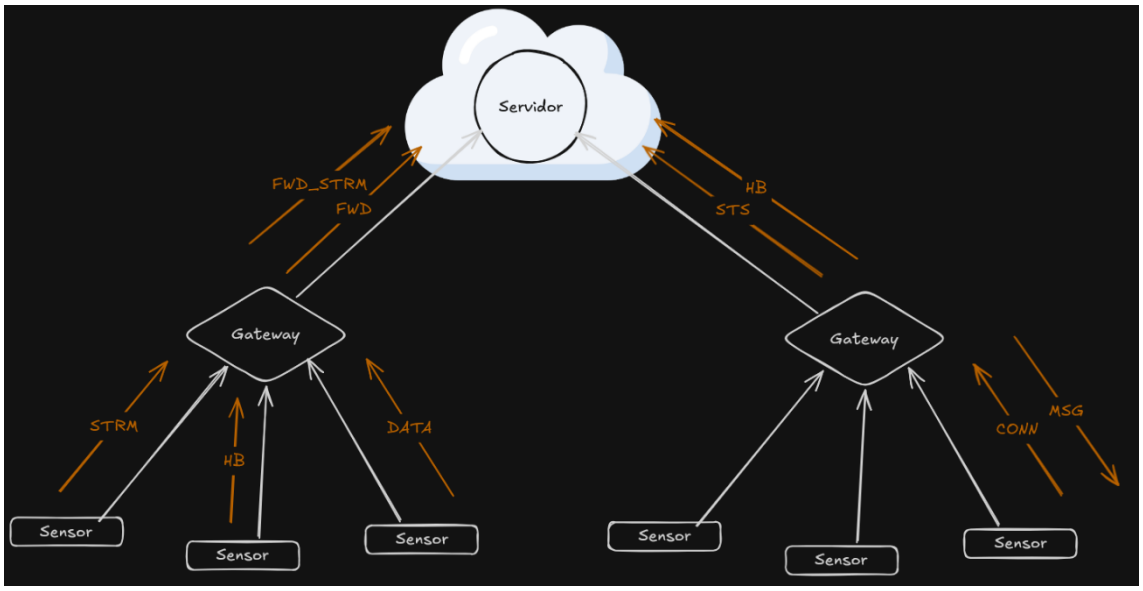

UrbanHealth
IoT Platform
Monitoring
Topology
Sensors Streaming
Analytics
Sensors Streaming
Last Reading
8:39:13 PM
Sensor ID (ex: S101)
+ Watch Transmission
REC | S101
REC | S102
REC | S103


UrbanHealth
IoT Platform
Monitoring
Topology
Sensors Streaming
Analytics
Analytics
Last Reading
8:40:48 PM
Zone
ZONA_1
Data Type
NOISE
Sensor (Optional)
S101
Calculate Statistics
Average
63.18
Median
61.5
Std Dev
14.83
Minimum
40
Maximum
89.8
Total Readings
214

```
C:\3ano\2semestre\SD\SD-TP

[GATEWAY] Active on port 5000
==================================================
 Gateway Menu. Available commands:
 -> DISCONN (to shutdown gateway in a clean way)
==================================================

[GATEWAY-VIDEO] UDP Listener listening on port 5002...
[WEB] Web Server active. Ready to stream
[GATEWAY] Connected to Server!
[GATEWAY] New sensor detected. Initiating handler...
[RECEIVED] Command: CONN by S101
[CONFIG] Sensor S101 connected. DataTypes updated to: TEMP,HUM,PM2,CO2,
NOISE,UV,VIDEO
[CONFIG] File sensores.csv updated with success.
[AUTH] Sensor S101 authorized in ZONA_TESTE
[FORWARD] Sensor S101 connected to gateway .
[RECEIVED] Command: DATA by S101
[DATA] S101 sent 20.1 (TEMP)
[RECEIVED] Command: HB by S101
[CACHE-DEBUG] Lendo pendentes do SQLite...
[CACHE-DEBUG] Total de registos pendentes encontrados: 0
[BATCHING] Pacote de 1 leituras de S101 enviado.
[RECEIVED] Command: DATA by S101
[DATA] S101 sent 82.4 (NOISE)
[RECEIVED] Command: HB by S101
[RECEIVED] Command: DATA by S101
[DATA] S101 sent 833.1 (CO2)
[RECEIVED] Command: DATA by S101
[DATA] S101 sent 20.3 (TEMP)
```

```
C:\3ano\2semestre\SD\SD-TP

[SYSTEM] Starting Sensor S101 connecting to 127.0.0.1:5000...
==================================================
 Interactive Menu. Available commands:
 -> DATA <TYPE> <VALUE> (e.g., DATA HUM 65.2)
 -> STRM START (to request video transmission)
 -> STRM STOP (to end video transmission)
 -> DISCONN (to gracefully shutdown)
==================================================

[SENSOR] Trying to connect to Gateway...
[SENSOR] Connected to Gateway!
[SENSOR] Sent CONN, waiting for response...
[AUTH] Success! Operating in zone: ZONA_TESTE
[DATA] TEMP: 20.1
[DATA] NOISE: 82.4
[DATA] CO2: 833.1
[DATA] TEMP: 20.3
[DATA] UV: 9.1
[SENSOR] High UV detected! Requesting video...
[STREAM] Gateway authorized START. Sending bytes...
[DATA] UV: 2.5
[DATA] NOISE: 61.9
[DATA] NOISE: 77.5
[DATA] HUM: 64.1
[DATA] PM2: 34.5
[STREAM] Environment stabilized for 30s. Stopping video...
[STREAM] Gateway authorized STOP. Video paused.
[DATA] HUM: 40.8
```

```
C:\3ano\2semestre\SD\SD-TP

[SERVER] Listening on port 5001
[DB] Database SQLite initialized with Single Table, Indexes, WAL mode
and Bulk-Insert Channel.
[SERVER-VIDEO] UDP Listener active on port 5003...
[WEB] Dashboard Ready: http://localhost:8081/
[SERVER] New connection detected (Gateway)!

[RECEIVED] Command: STS | GID: G101
   -> Status Message: ONLINE | SID:

[RECEIVED] Command: HB | GID: G101

[RECEIVED] Command: STS | GID: G101
   -> Status Message: ONLINE | SID: S101
[STATUS] Sensor S101 state: ONLINE

[RECEIVED] Command: HB | GID: G101

[RECEIVED] Command: HB | GID: G101

[RECEIVED] Command: FWD | GID: G101

   -> [BATCHING] Gateway G101 sent batch of data from Sensor: S101
   -> Zone: ZONA_TESTE | Type: TEMP
   -> Total of readings: 1

[RECEIVED] Command: FWD | GID: G101

   -> [BATCHING] Gateway G101 sent batch of data from Sensor: S101
   -> Zone: ZONA_TESTE | Type: PM2
```



Estrutura das mensagens

```
CMD: CONN
ID: S102
DATA_TYPES: TEMP,HUM,PM2,CO2,NOISE,UV
<EOF>
```

```
CMD: DATA
SID: S101
TYPE: CO2
VALUE: 450.5
<EOF>
```

```
CMD: STRM
SID: S101
ACTION: START/STOP
<EOF>
```

```
CMD: STS
GID: G102
SID: S104
STATUS: OFFLINE/ONLINE/MAINTENANCE
<EOF>
```

```
CMD: FWD_STRM
GID: G101
SID: S101
ACTION: START (ou STOP)
<EOF>
```

```
CMD: STRM
SID: S101
GID: G101
TYPE: DATA
PART: 1
TOTAL: 5
<EOF>
[...BYTES_HERE...]
```

```
CMD: HB
SID: S103
<EOF>
```

```
CMD: HB
GID: G102
<EOF>
```

```
CMD: MSG
TYPE: ERR
REF_CMD: CONN
REASON: UNKNOWN_SENSOR (ou MAINTENANCE)
<EOF>
```

```
CMD: FWD
GID: G101
SID: S101
ZONE: ZONE_3
TYPE: CO2
BATCH_COUNT: 5
RAW_PAYLOAD: [{"Timestamp":"...", "Value":"450.5"},...]
<EOF>
```

```
CMD: MSG
TYPE: ACK
REF_CMD: CONN
ZONE: ZONE_1
<EOF>
```